

Test Scenarios & Analysis

Scenario A: Basic Mixed Workload

- **Goal:** To test the standard behavior of algorithms with varied arrival and burst times.
- **Analysis:** SJF prioritized shorter tasks, leading to the lowest average waiting time. Priority scheduling followed the assigned ranks, which caused a slight increase in the overall waiting time.
- **Conclusion:** SJF is optimal for reducing average delays in a normal workload.

The screenshot shows a web-based simulation interface titled "SJF vs Priority Comparison". It features a navigation bar with "Input", "Gantt Charts", "Results", and "Analysis & Conclusion". The "Scheduling Mode" section has two radio buttons: "SJF Preemptive (SRTF)" (selected) and "Priority Preemptive". Below this is the "Process Input Table" with the following data:

PID	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	4
P5	4	2	5

At the bottom, there are four buttons: "+ Add Process" (green), "- Remove Selected" (red), "Clear All" (grey), and "Run Simulation" (green).



SJF vs Priority Comparison

Input | Gantt Charts | Results | Analysis & Conclusion

SJF — Average Metrics

Avg WT	Avg TAT	Avg RT
8.20	13.80	8.20

Priority — Average Metrics

Avg WT	Avg TAT	Avg RT
11.40	17.00	11.40

SJF — Per-Process Metrics

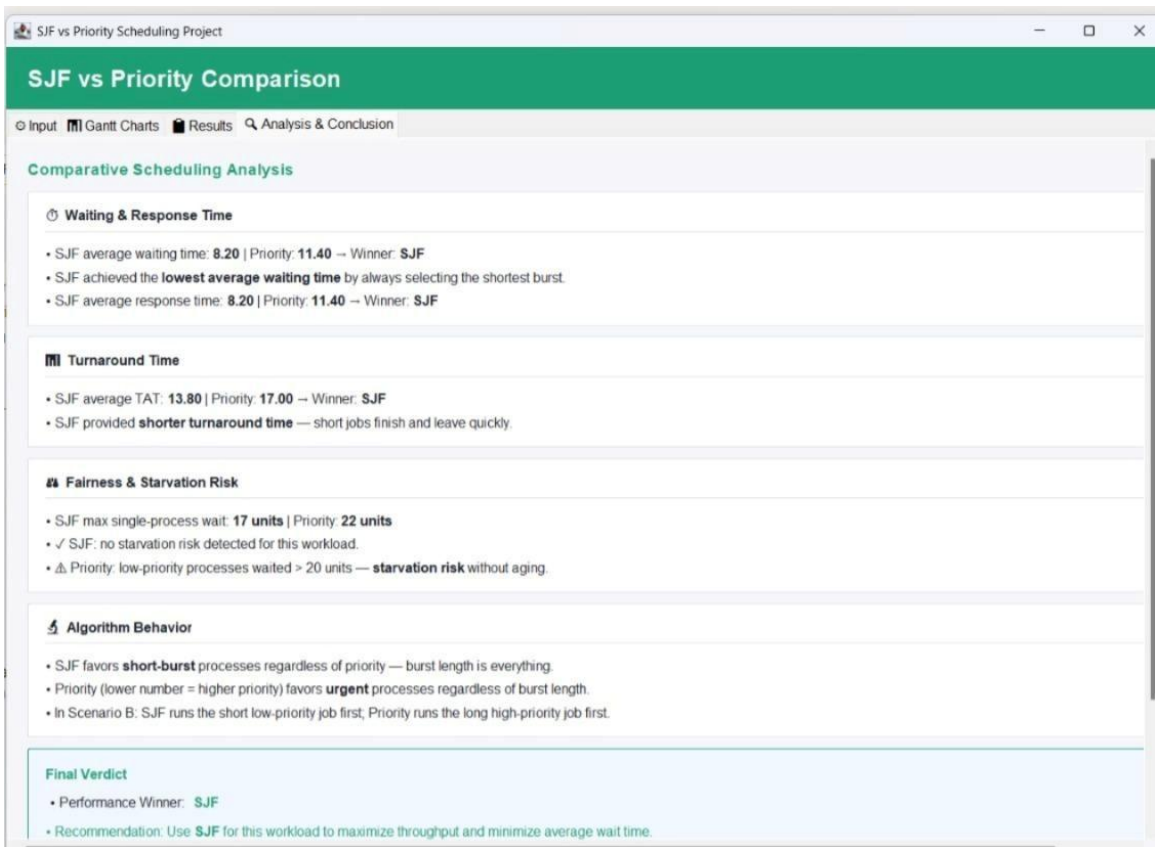
PID	AT	BT	Priority	FT	WT	TAT	RT
P1	0	8	2	8	0	8	0
P5	4	2	5	10	4	6	4
P2	1	4	1	14	9	13	9
P4	3	5	4	19	11	16	11
P3	2	9	3	28	17	26	17

Priority — Per-Process Metrics

PID	AT	BT	Priority	FT	WT	TAT	RT
P1	0	8	2	8	0	8	0
P2	1	4	1	12	7	11	7
P3	2	9	3	21	10	19	10
P4	3	5	4	26	18	23	18
P5	4	2	5	28	22	24	22

★ Comparison Summary

Average Waiting Time:	SJF: 8.20	Priority: 11.40	Winner: ✓ SJF
Average Turnaround Time:	SJF: 13.80	Priority: 17.00	Winner: ✓ SJF
Average Response Time:	SJF: 8.20	Priority: 11.40	Winner: ✓ SJF
SJF max single-process wait : 17 units			
Priority max single-process wait : 22 units			



SJF vs Priority Scheduling Project

SJF vs Priority Comparison

Input

Gantt Charts

Results

Analysis & Conclusion

Scheduling Mode

☒ SJF Preemptive (SRTF)

☐ Priority Preemptive

Process Input Table

PID	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	4
P5	4	2	5

+ Add Process

— Remove Selected

Clear All

▶ Run Simulation

SJF vs Priority Scheduling Project

SJF vs Priority Comparison

Input

Gantt Charts

Results

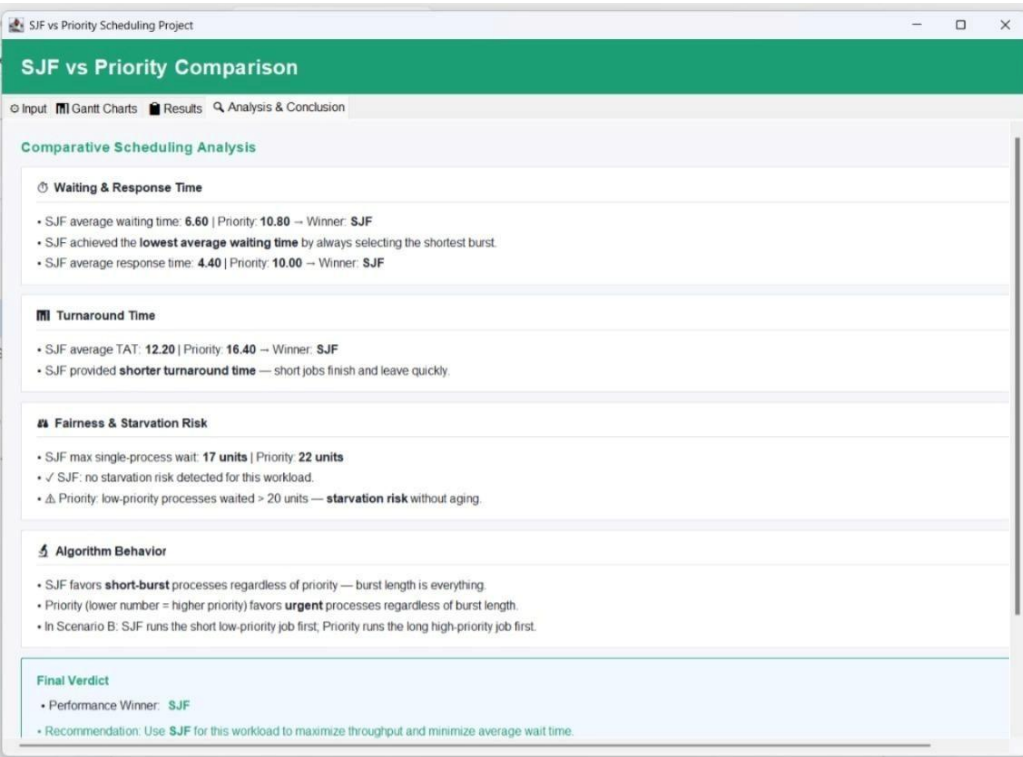
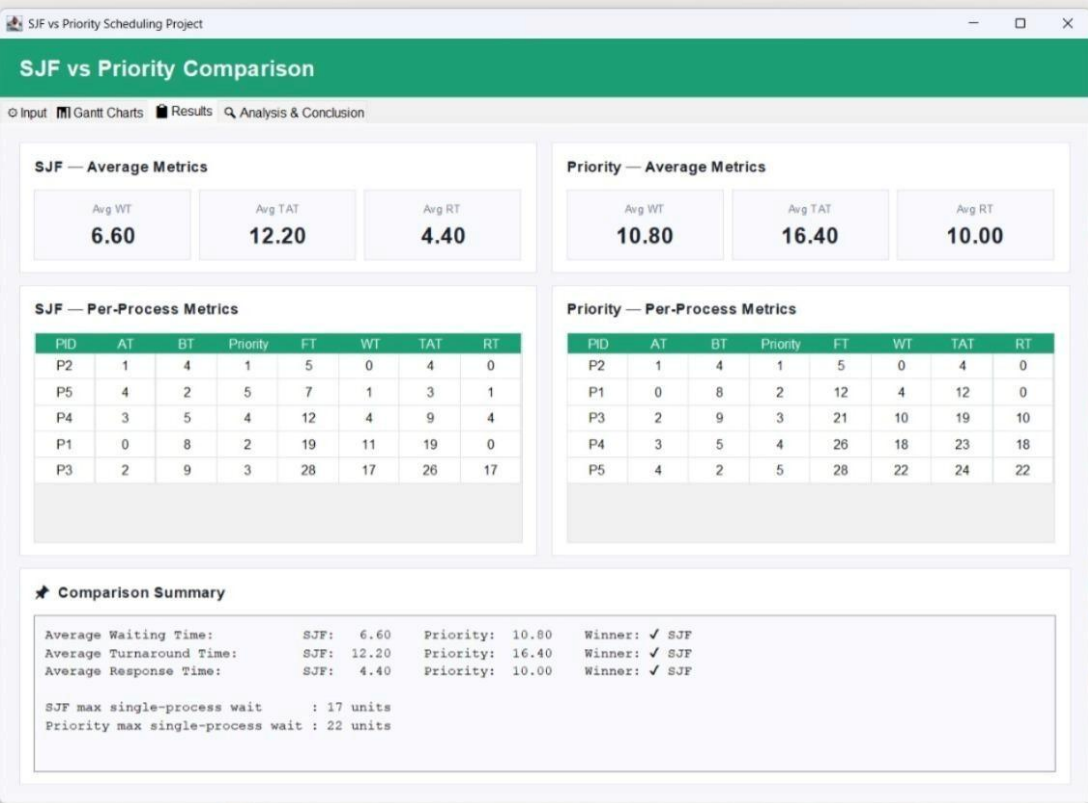
Analysis & Conclusion

Gantt Chart — Shortest Job First (SJF, Preemptive - SRTF)

0 1 5 7 12 19

Gantt Chart — Priority Scheduling (Preemptive)

0 1 5 12 21 26



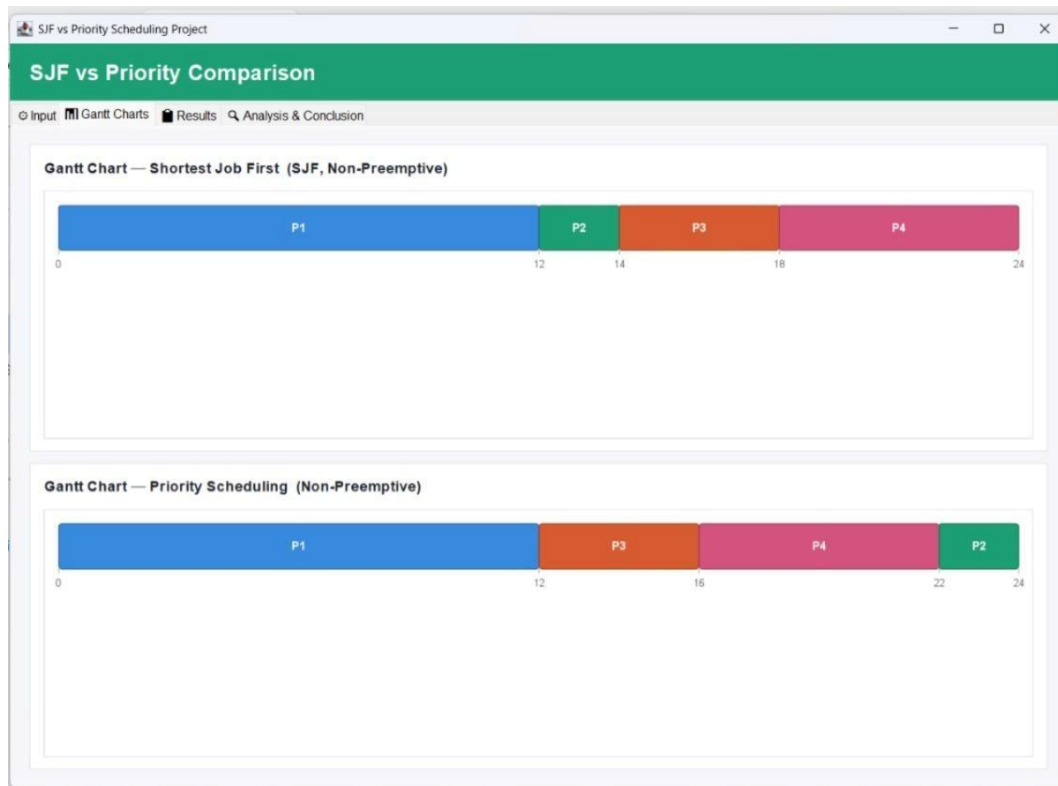
Scenario B: Conflict (Short-Burst vs. High-Priority)

- **Goal:** To observe the system's decision when a very short job has a low priority compared to a long job with high priority.
- **Analysis:** In Priority mode, the high-priority process took control first despite its length. In SJF, the short job was executed first to improve efficiency.
- **Conclusion:** This scenario highlights the sacrifice of system speed for the sake of task importance in Priority Scheduling.

The screenshot displays a web application titled "SJF vs Priority Scheduling Project". The main heading is "SJF vs Priority Comparison". Below this, there are tabs for "Input", "Gantt Charts", "Results", and "Analysis & Conclusion". The "Scheduling Mode" section has two radio buttons: "SJF Preemptive (SRTF)" (selected) and "Priority Preemptive". The "Process Input Table" contains the following data:

PID	Arrival Time	Burst Time	Priority
P1	0	12	1
P2	2	2	5
P3	3	4	2
P4	5	6	3

At the bottom, there are buttons for "+ Add Process", "- Remove Selected", "Clear All", and "Run Simulation".



SJF vs Priority Scheduling Project

SJF vs Priority Comparison

Input Gantt Charts Results Analysis & Conclusion

SJF — Average Metrics								Priority — Average Metrics							
Avg WT				Avg TAT				Avg WT				Avg TAT			
8.50				14.50				10.00				16.00			
Avg RT				Avg RT				Avg RT				Avg RT			
8.50								10.00				10.00			

SJF — Per-Process Metrics								Priority — Per-Process Metrics							
PID	AT	BT	Priority	FT	WT	TAT	RT	PID	AT	BT	Priority	FT	WT	TAT	RT
P1	0	12	1	12	0	12	0	P1	0	12	1	12	0	12	0
P2	2	2	5	14	10	12	10	P3	3	4	2	16	9	13	9
P3	3	4	2	18	11	15	11	P4	5	6	3	22	11	17	11
P4	5	6	3	24	13	19	13	P2	2	2	5	24	20	22	20

★ Comparison Summary

Average Waiting Time:	SJF: 8.50	Priority: 10.00	Winner: ✓ SJF
Average Turnaround Time:	SJF: 14.50	Priority: 16.00	Winner: ✓ SJF
Average Response Time:	SJF: 8.50	Priority: 10.00	Winner: ✓ SJF
SJF max single-process wait : 13 units			
Priority max single-process wait : 20 units			

SJF vs Priority Comparison

Input Gantt Charts Results Analysis & Conclusion

Comparative Scheduling Analysis

⌚ Waiting & Response Time

- SJF average waiting time: **8.50** | Priority: **10.00** → Winner: **SJF**
- SJF achieved the **lowest average waiting time** by always selecting the shortest burst.
- SJF average response time: **8.50** | Priority: **10.00** → Winner: **SJF**

📊 Turnaround Time

- SJF average TAT: **14.50** | Priority: **16.00** → Winner: **SJF**
- SJF provided **shorter turnaround time** — short jobs finish and leave quickly.

⚖️ Fairness & Starvation Risk

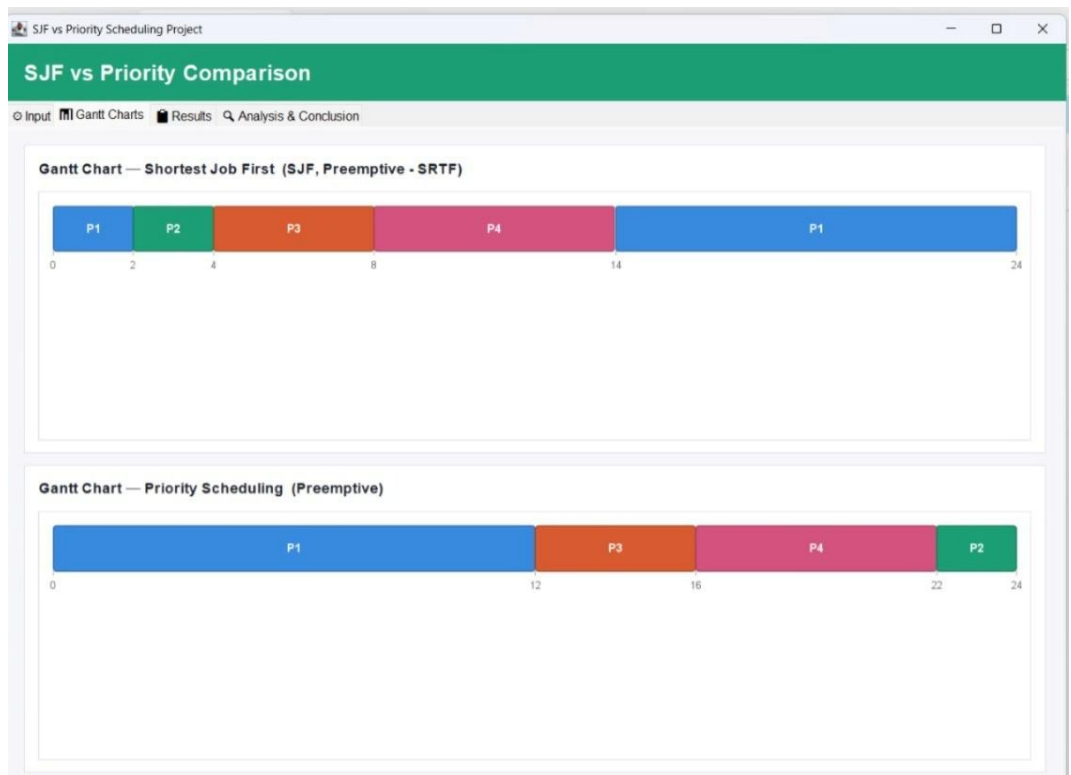
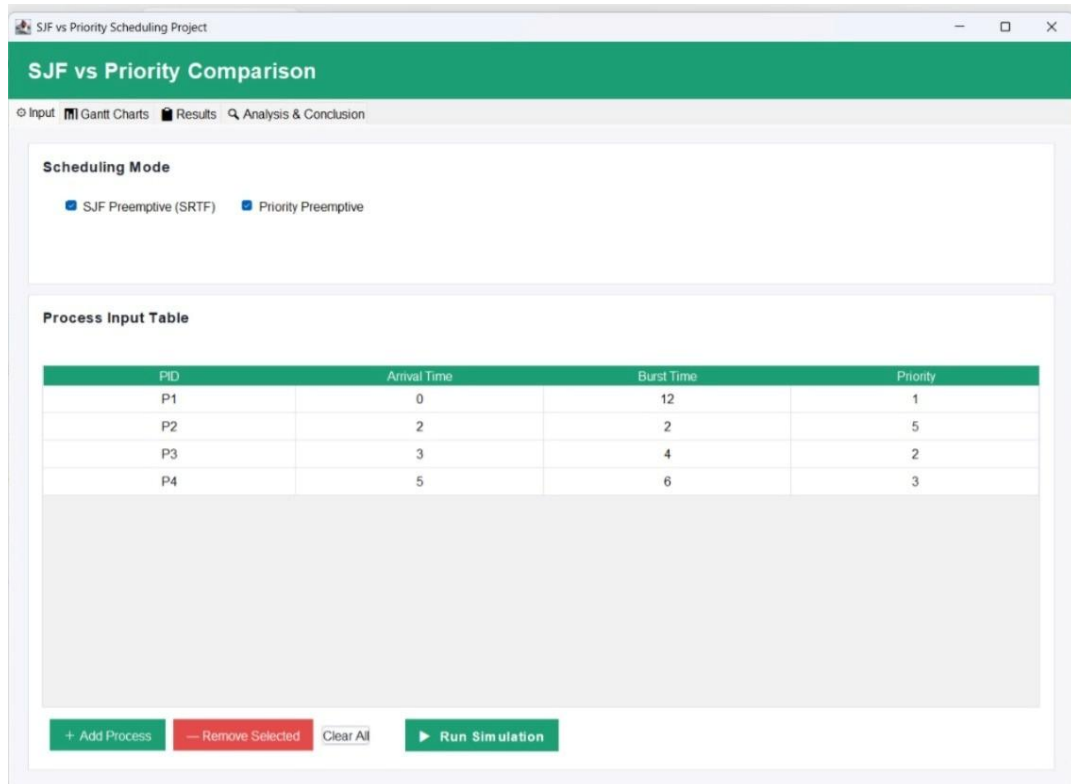
- SJF max single-process wait: **13 units** | Priority: **20 units**
- ✓ SJF: no starvation risk detected for this workload.
- ✓ Priority: no starvation risk detected for this workload.

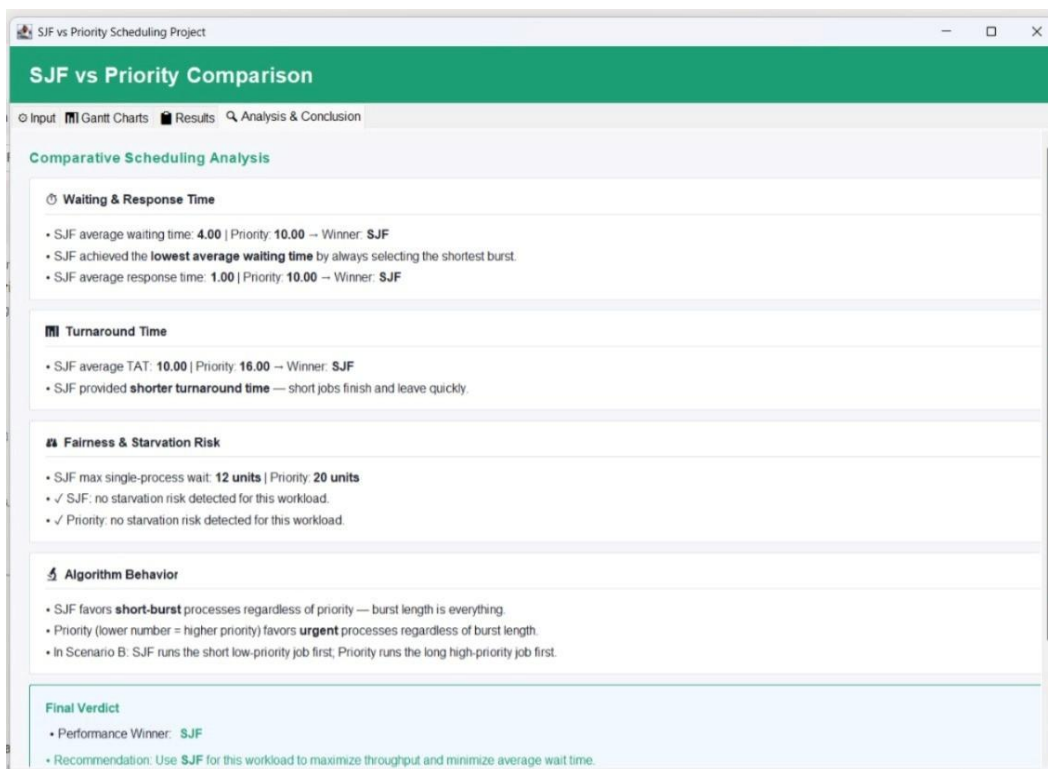
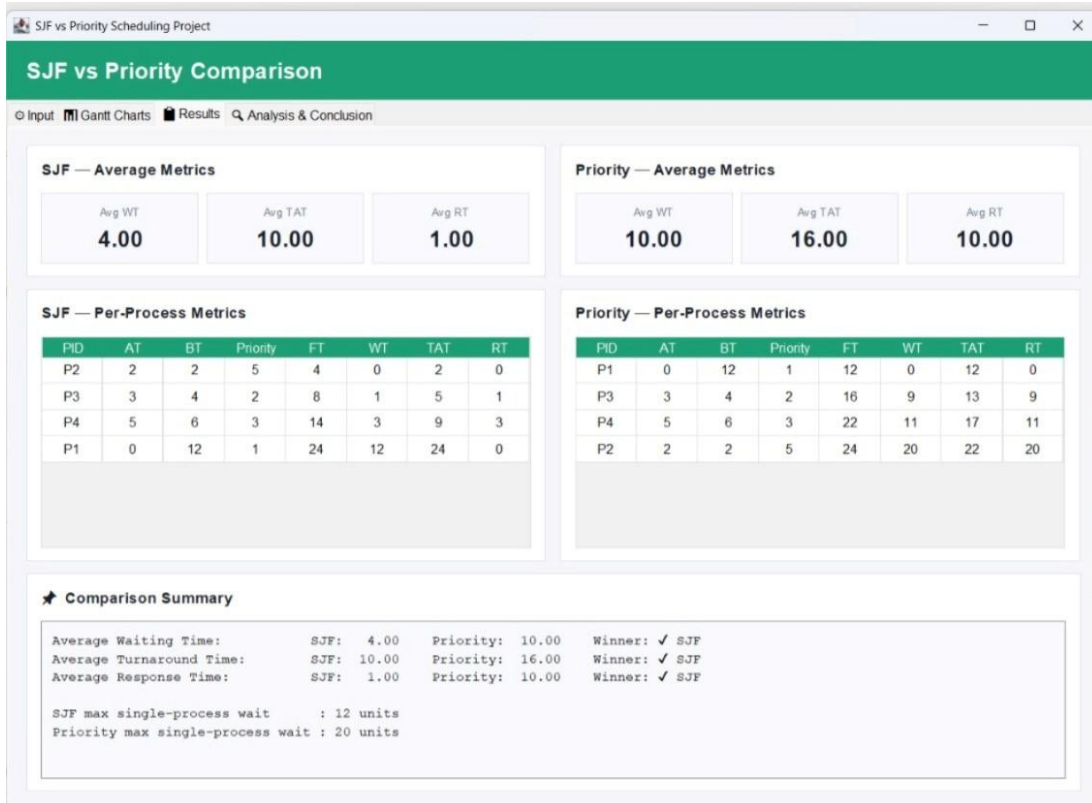
🔍 Algorithm Behavior

- SJF favors **short-burst** processes regardless of priority — burst length is everything.
- Priority (lower number = higher priority) favors **urgent** processes regardless of burst length.
- In Scenario B: SJF runs the short low-priority job first, Priority runs the long high-priority job first.

Final Verdict

- Performance Winner: **SJF**
- Recommendation: Use **SJF** for this workload to maximize throughput and minimize average wait time.





"In Scenario B, Priority Scheduling caused process **P2** to wait for a long time despite its small burst time, because it had a low priority. Conversely, SJF prioritized smaller tasks to keep the average waiting time low."

Scenario C: Fairness & Starvation Case

- **Goal:** To detect "Starvation" where a process is delayed indefinitely.
- **Analysis:** In Priority Scheduling, the process with the lowest priority (highest number) suffered a massive delay (Starvation). SJF was fairer as it eventually processed all jobs based on their size.
- **Conclusion:** Priority scheduling requires "Aging" techniques to be fair, whereas SJF is naturally more balanced for smaller tasks.

SJF vs Priority Comparison

Input | Gantt Charts | Results | Analysis & Conclusion

Scheduling Mode

☐ SJF Preemptive (SRTF) ☐ Priority Preemptive

Process Input Table

PID	Arrival Time	Burst Time	Priority
P1	0	5	1
P2	0	5	2
P3	0	12	5
P4	0	2	1
P5	0	3	2

+ Add Process - Remove Selected Clear All Run Simulation



SJF vs Priority Scheduling Project

SJF vs Priority Comparison

Input Gantt Charts Results Analysis & Conclusion

SJF — Average Metrics

Avg WT	Avg TAT	Avg RT
6.40	11.80	6.40

Priority — Average Metrics

Avg WT	Avg TAT	Avg RT
7.80	13.20	7.80

SJF — Per-Process Metrics

PID	AT	BT	Priority	FT	WT	TAT	RT
P4	0	2	1	2	0	2	0
P5	0	3	2	5	2	5	2
P1	0	5	1	10	5	10	5
P2	0	5	2	15	10	15	10
P3	0	12	5	27	15	27	15

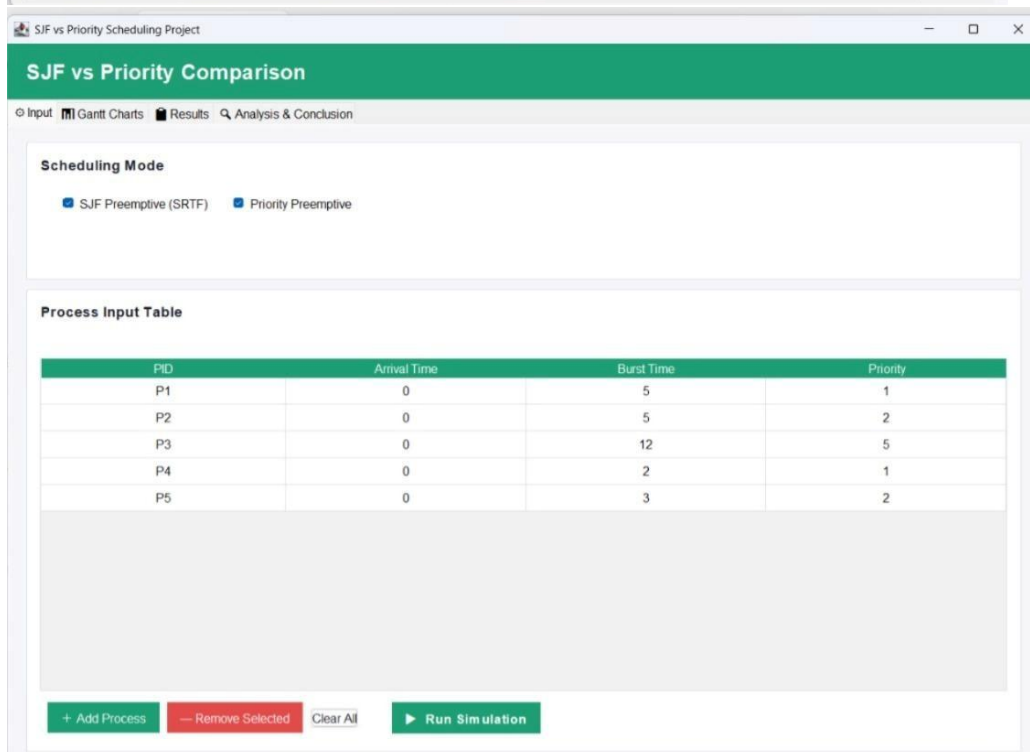
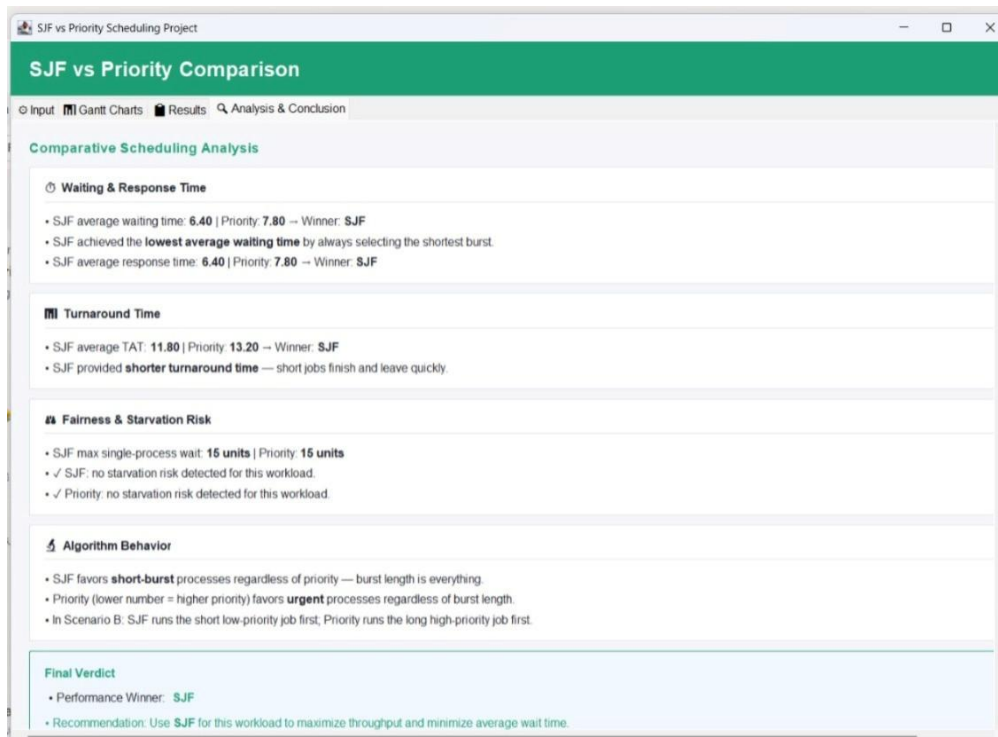
Priority — Per-Process Metrics

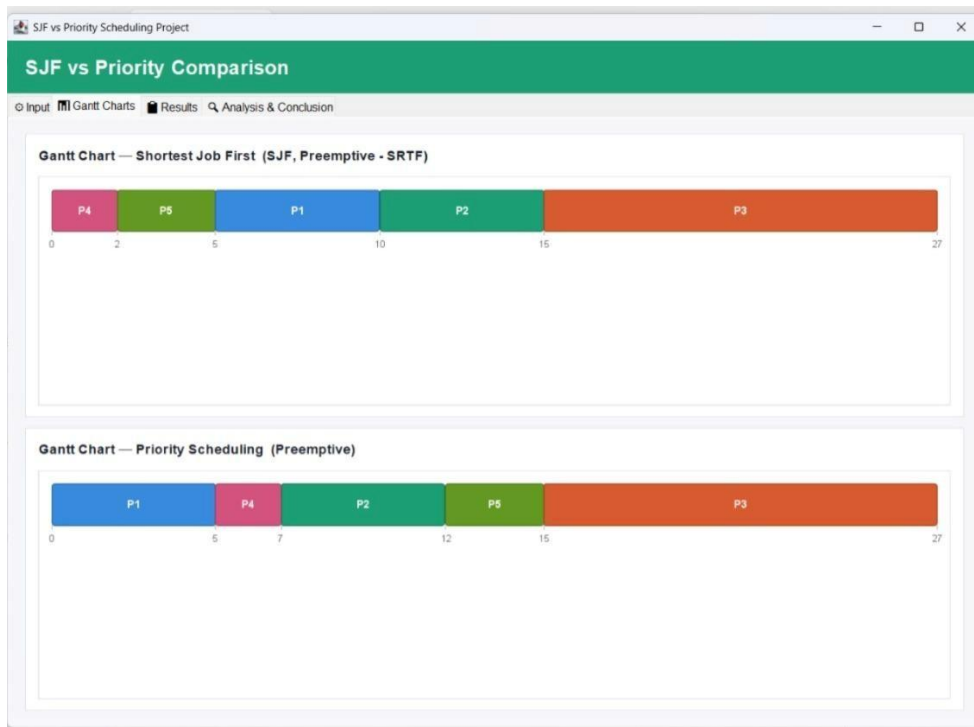
PID	AT	BT	Priority	FT	WT	TAT	RT
P1	0	5	1	5	0	5	0
P4	0	2	1	7	5	7	5
P2	0	5	2	12	7	12	7
P5	0	3	2	15	12	15	12
P3	0	12	5	27	15	27	15

★ Comparison Summary

Average Waiting Time:	SJF: 6.40	Priority: 7.80	Winner: ✓ SJF
Average Turnaround Time:	SJF: 11.80	Priority: 13.20	Winner: ✓ SJF
Average Response Time:	SJF: 6.40	Priority: 7.80	Winner: ✓ SJF

SJF max single-process wait : 15 units
Priority max single-process wait : 15 units





SJF vs Priority Scheduling Project

SJF vs Priority Comparison

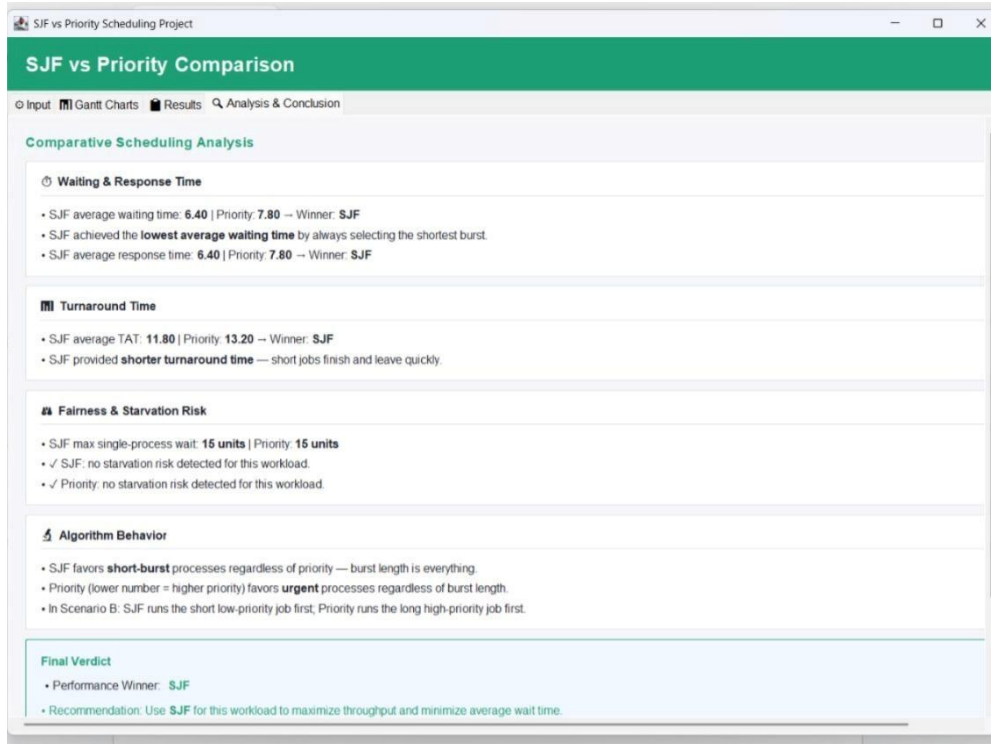
Input Gantt Charts Results Analysis & Conclusion

SJF — Average Metrics								Priority — Average Metrics							
Avg WT		Avg TAT		Avg RT				Avg WT		Avg TAT		Avg RT			
6.40		11.80		6.40				7.80		13.20		7.80			

SJF — Per-Process Metrics								Priority — Per-Process Metrics							
PID	AT	BT	Priority	FT	WT	TAT	RT	PID	AT	BT	Priority	FT	WT	TAT	RT
P4	0	2	1	2	0	2	0	P1	0	5	1	5	0	5	0
P5	0	3	2	5	2	5	2	P4	0	2	1	7	5	7	5
P1	0	5	1	10	5	10	5	P2	0	5	2	12	7	12	7
P2	0	5	2	15	10	15	10	P5	0	3	2	15	12	15	12
P3	0	12	5	27	15	27	15	P3	0	12	5	27	15	27	15

★ Comparison Summary

Average Waiting Time:	SJF: 6.40	Priority: 7.80	Winner: ✓ SJF
Average Turnaround Time:	SJF: 11.80	Priority: 13.20	Winner: ✓ SJF
Average Response Time:	SJF: 6.40	Priority: 7.80	Winner: ✓ SJF
SJF max single-process wait : 15 units			
Priority max single-process wait : 15 units			



"Scenario C demonstrates the **Starvation** issue in Priority Scheduling. Process **P3** suffers from an unfair delay because it has the lowest priority, even though it arrived at the same time as others. SJF handled the workload based on task size, showing better overall fairness for the system."

Scenario D: Validation & Error Handling

- **Goal:** To ensure the robustness of the software against invalid user inputs.
- **Analysis:** When entering negative values or non-numeric characters (e.g., "abc"), the system successfully blocked the execution and displayed a clear error message.
- **Conclusion:** The application is robust and prevents system crashes by validating all inputs before processing.

SJF vs Priority Scheduling Project

SJF vs Priority Comparison

Input

Gantt Charts

Results

Analysis & Conclusion

Scheduling Mode

☐ SJF Preemptive (SRTF)

☐ Priority Preemptive

Process Input Table

⚠ Process P3: Arrival Time must be ≥ 0.

PID	Arrival Time	Burst Time	Priority
P1	0	2	1
P2	0	3	2
P3	-2	12	5
P4	0	2	1

+ Add Process

— Remove Selected

Clear All

▶ Run Simulation

SJF vs Priority Scheduling Project

SJF vs Priority Comparison

Input

Gantt Charts

Results

Analysis & Conclusion

Scheduling Mode

☐ SJF Preemptive (SRTF)

☐ Priority Preemptive

Process Input Table

⚠ Process P2: Burst Time must be an integer (got "abc").

PID	Arrival Time	Burst Time	Priority
P1	0	2	1
P2	0	abc	2
P3	-2	12	5
P4	0	2	1

+ Add Process

— Remove Selected

Clear All

▶ Run Simulation

"This scenario tests the robustness of the application. The system successfully detects invalid inputs (such as non-numeric values or negative times) and prevents the simulation from running, providing clear feedback to the user."